

# Assessment - 4

classmate

Date \_\_\_\_\_

Page \_\_\_\_\_

## ◦ Aim :-

- To write and simulate verilog codes SR, JK, D and T flip flops.
- To understand their working and verify outputs through simulation.

## ◦ Theory :-

- ① Combinational (VS) Sequential Circuits
- ② Latch (VS) flip flops
- ③ Triggering signal in Sequential circuits
- ④ flip-flops.

## ◦ Tools Required :-

- Software : xilinx ISE / vivado / ModelSim
- Language : Verilog HDL
- Hardware : FPGA Kit
- Computer system with simulation environment.



Date  
23-Sep-25

## Assessment - 4

classmate

Date

Page

### • SR-flip flop :-

```
module sr_ff (clk, rst, s, r, q);  
    input clk, rst, s, r;  
    output reg q;  
    always @ (posedge clk or posedge rst)  
    begin  
        if (rst == 1)  
            q <= 0;  
        else if (s == 0 && r == 0)  
            q <= q;  
        else if (s == 0 && r == 1)  
            q <= 0;  
        else if (s == 1 && r == 0)  
            q <= 1;  
        else if (s == 1 && r == 1)  
            q <= ~q;  
        end  
    endmodule
```

### • module tb ();

```
    reg clk, rst, s, r;  
    wire q;  
    sr_ff uut (clk, rst, s, r, q);  
    always begin  
        #10 clk = ~clk;  
    end  
    initial  
    begin  
        clk = 0; rst = 0; s = 0; r = 0;  
        #5 rst = 1;  
        #5 rst = 0; s = 1; r = 0;  
        #20 s = 0; r = 1; s = 0; r = 0;  
        #20 rst = 1; rst = 0;  
    end
```

O/P verified  
23/8/25  
24BCE0403



# JK - flip flop

classmate

Date

Page

```
module JK_ff (clk, rst, j, k, q);  
    input clk, rst, j, k;  
    output reg q;  
    always @ (posedge clk or posedge rst)  
    begin  
        if (rst == 1)  
            q <= 0;  
        else if (j == 0 && k == 0)  
            q <= q;  
        else if (j == 0 && k == 1)  
            q <= 0;  
        else if (j == 1 && k == 0)  
            q <= 1;  
        else if (j == 1 && k == 1)  
            q <= ~q;  
        end  
    endmodule
```

```
module tb();  
    reg clk, rst, j, k;  
    wire q;  
    JK_ff uut (clk, rst, j, k, q);  
    always begin  
        #10 clk = ~clk;  
    end  
    initial  
    begin  
        clk = 0; rst = 0; j = 0; k = 0;  
        #5 rst = 1;  
        #5 rst = 0; j = 0; k = 0;  
        #25 j = 0; k = 1;  
        #25 j = 1; k = 1;  
        #25 j = 1; k = 0;  
    end  
endmodule
```

o/p verified  
23/8/25  
24BCE0403



# T-flip flop

classmate

Date

Page

```
module t_ff (clk, rst, t, q);  
    input clk, rst, t;  
    output reg q;  
    always @ (posedge clk or posedge rst)  
    begin  
        if (rst == 1)  
            q <= 0;  
        else if (t == 0)  
            q <= q;  
        else  
            q <= ~q;  
        end  
    endmodule
```

```
module tb ();  
    reg clk, rst, t;  
    wire q;  
    t_ff uut (clk, rst, t, q);  
    always begin  
        #10 clk = ~clk;  
        end  
    initial  
    begin
```

O/P verified

23/8/23

24BC60403

```
    clk = 0; rst = 0; t = 0;  
    #5 rst = 1;  
    #5 rst = 0; t = 0;  
    #15 t = 1;  
    #10 t = 0;  
    #15 t = 1;  
    #5 rst = 1;  
    #5 rst = 0; t = 1;  
    # stop  
    end  
endmodule
```



# D-flip flop

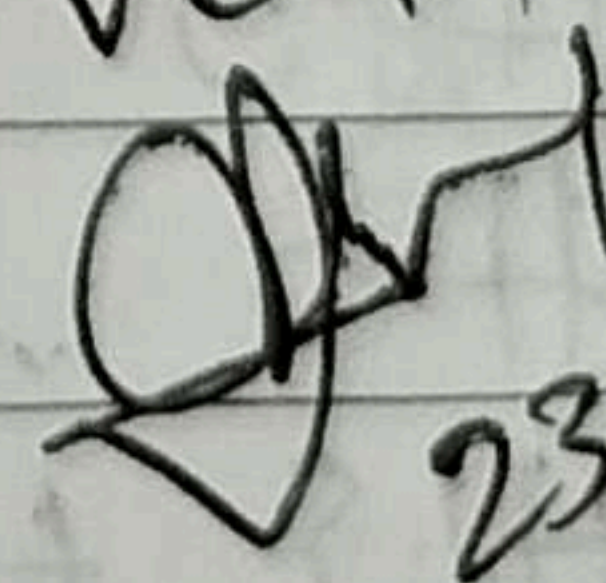
classmate

Date \_\_\_\_\_  
Page \_\_\_\_\_

```
• module d_ff (clk, rst, d, q);  
  input clk, rst, d;  
  output reg q;  
  always @ (posedge clk or posedge rst)  
  begin  
    if (rst == 1)  
      q <= 0;  
    else  
      q <= d;  
    end  
  end  
endmodule
```

```
• module tb ();  
  reg clk, rst, d;  
  wire q;  
  d_ff uut (clk, rst, d, q);  
  always begin  
    #10 clk = ~clk;  
  end  
  initial  
  begin  
    clk = 0; rst = 0; d = 0;  
    #5 rst = 1;  
    #5 rst = 0; d = 0;  
    #15 d = 1;  
    #10 d = 0;  
    #15 d = 1;  
    #5 rst = 1;  
    #5 rst = 0; d = 1;  
    $stop  
  end  
endmodule
```

o/p verified

  
23/8/23

24BCE0403



\* Answer the following :-

① Difference b/w combinational And Sequential Circuits.

Ans combinational :- output depends only on present inputs (no memory)

Sequential :- output depends on present input and past history.  
(uses memory)

② Difference b/w latch and flip flop.

Ans latch :- level-triggered (works when input / clock is at a level).

flip flop :- edge-triggered (works only on clock transition - rising or falling edge).

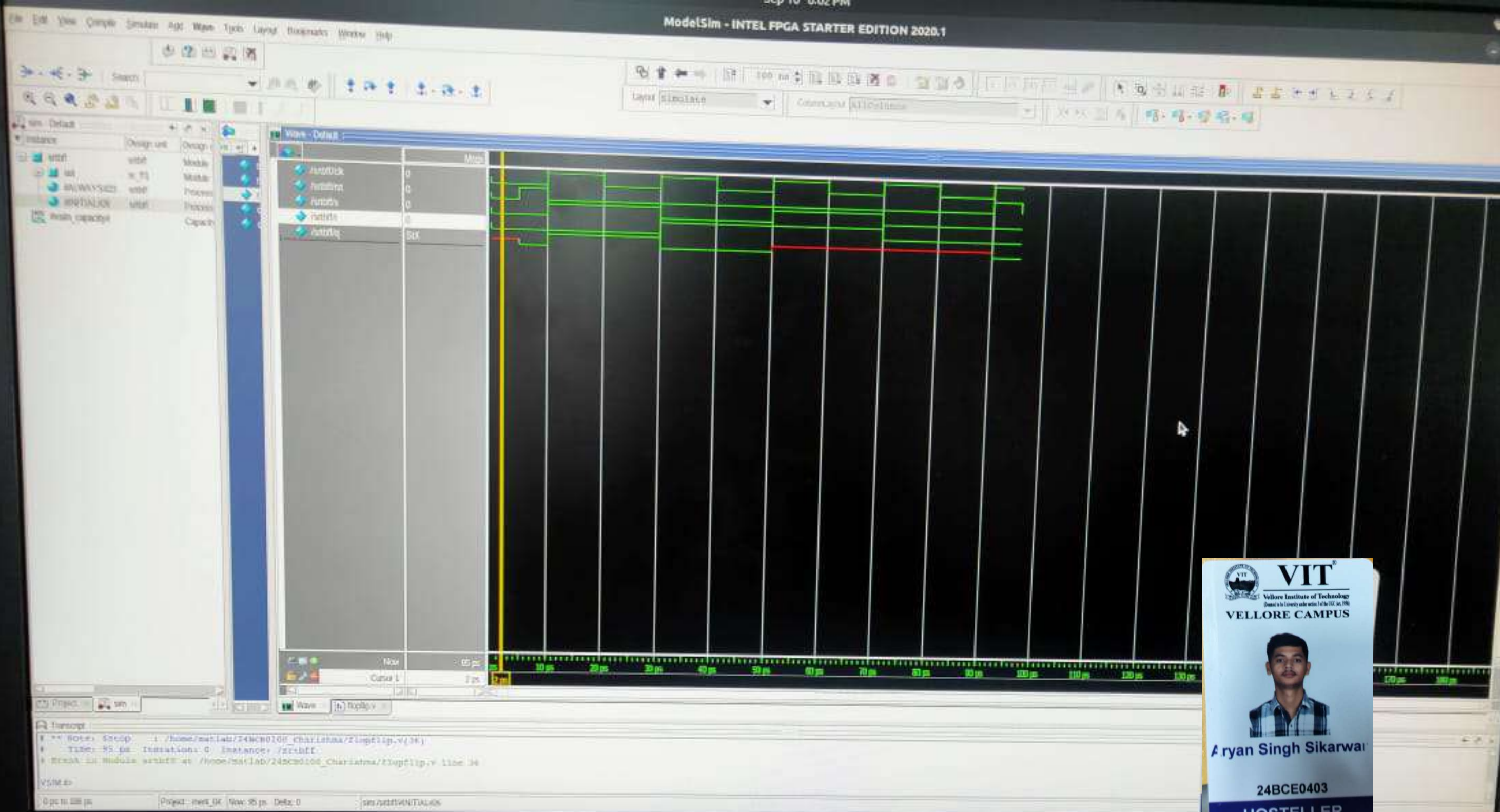
③ what are the different triggering signals in Sequential Circuits.

- positive edge triggering (rising edge of clock)

- negative edge triggering (falling edge of clock)

- level triggering (high level or low level of clock)









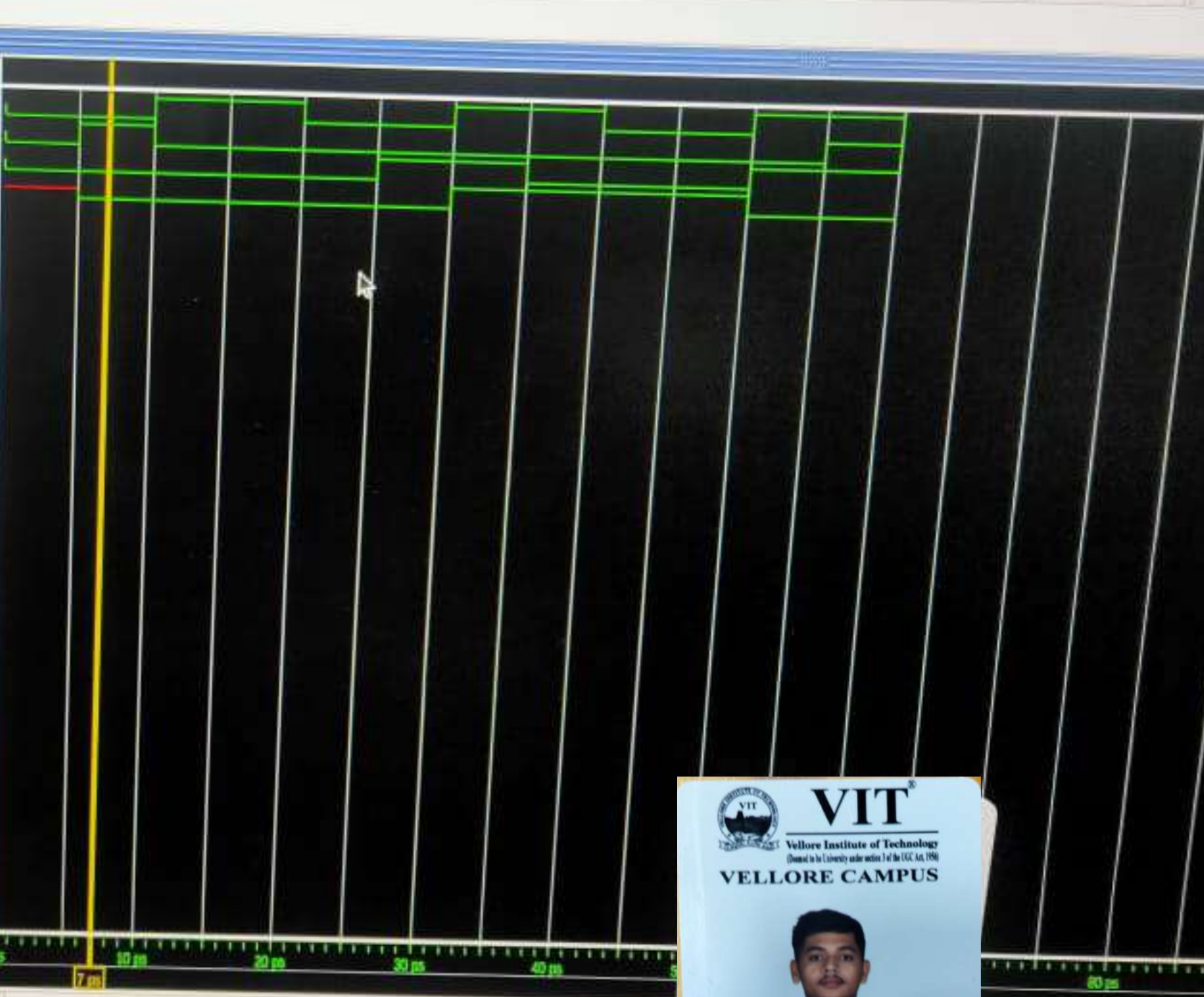


File Edit View Compile Simulate Add Wave Tools Layout Bookmarks Window Help



| Instance       | Design unit | Design i |
|----------------|-------------|----------|
| ttt_tbt        | ttt_tbt     | Module   |
| out            | ttt_tbt     | Module   |
| #ALWAYS#19     | ttt_tbt     | Process  |
| #INITIAL#22    | ttt_tbt     | Process  |
| #vsm_capacity# | ttt_tbt     | Capach   |

| Wave - Default | Meas |
|----------------|------|
| /ttt_tbt/clk   | 0    |
| /ttt_tbt/reset | 1    |
| /ttt_tbt/out   | 0    |
| /ttt_tbt/q     | Sx0  |



Transcript

```
# ** Mon: 5:00pm : /home/malab/24BCE0403_Charishma/tff.v(32)
# Time: 60 ps Iteration: 0 Instance: /ttt_tbt
# Break in Module ttt_tbt at /home/malab/24BCE0403_Charishma/tff.v line 32
```

VSM 2>

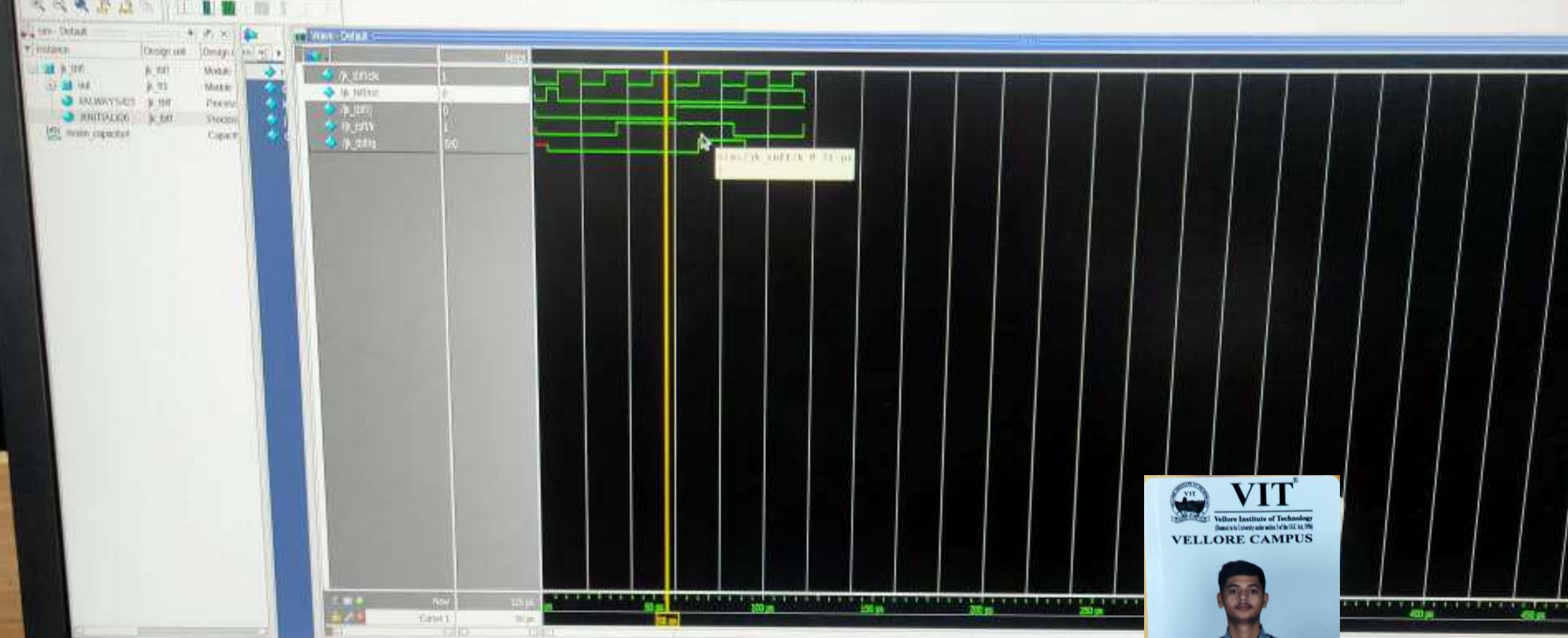
0 ps to 120 ps Project: eng0403 Now: 60 ps Defa: 0 /ttt\_tbt

 **VIT**  
Vellore Institute of Technology  
(Chartered in the University under section 3 of the UGC Act, 1956)  
**VELLORE CAMPUS**

  
**Aryan Singh Sikarwal**

**24BCE0403**  
**HOSTELLER**





Timing Diagram

File: /home/aryan/Projects/ModelSim/24BCE0403/24BCE0403.prj  
Device: Cyclone IV EP4K10K10-1  
Simulation: 0 - 500 ns  
Signal: output[0]

0 ps to 500 ps | Project: 24BCE0403 | View: 100 ps | Data: 0 | /p\_20/

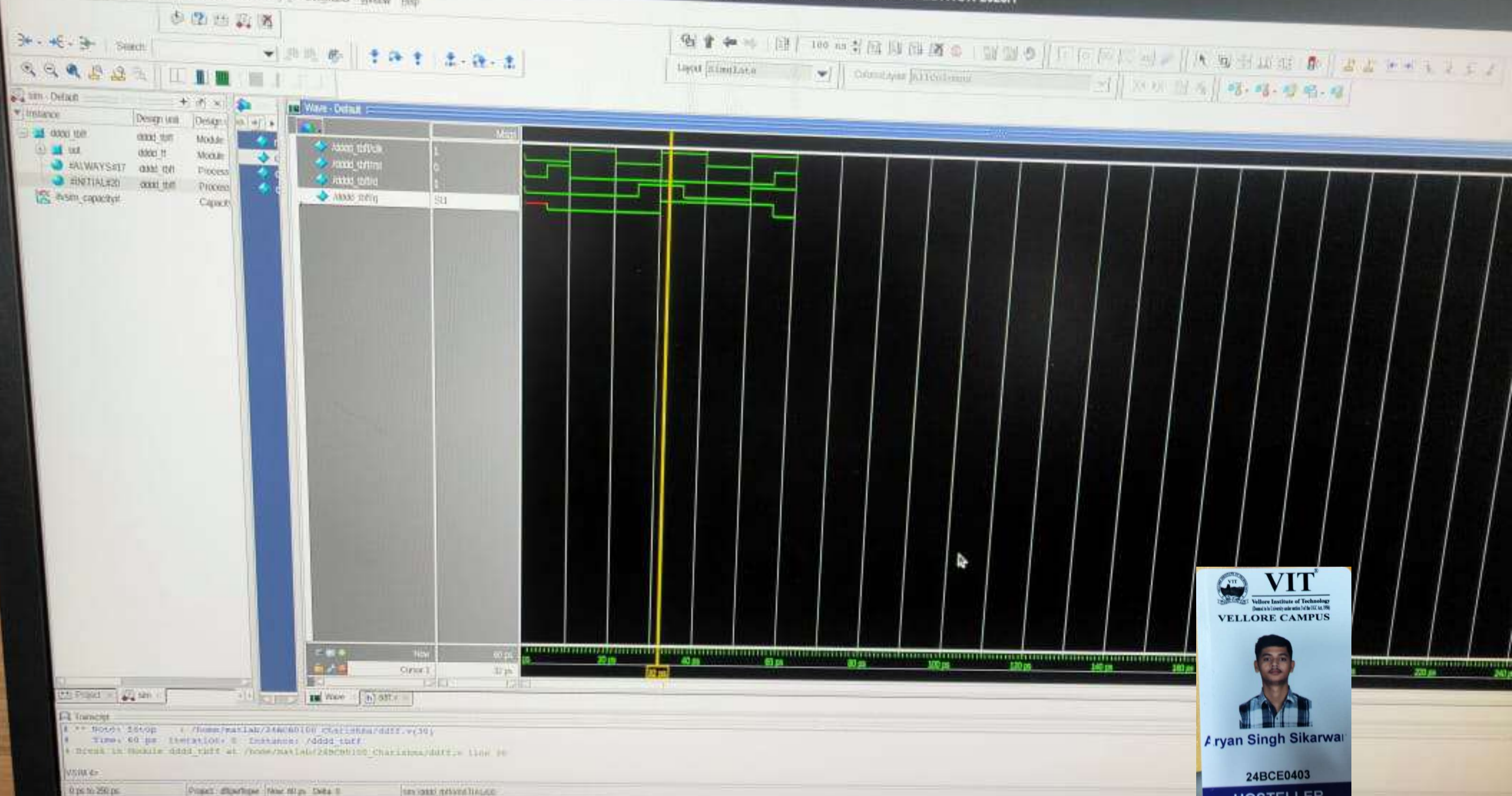
**VIT**  
Vellore Institute of Technology  
(Deemed to be University under section 3 of the UGC Act, 1956)  
VELLORE CAMPUS



**Aryan Singh Sikarwal**

24BCE0403  
HOSTELLER







Ln#

```
1  module d_ff(clk,rst,d,q);
2      input clk,rst,d;
3      output reg q;
4      always@(posedge clk or posedge rst)
5      begin
6          if(rst==1)
7              q<=0;
8          else
9              q<=d;
10     end
11     endmodule
12
13  module tb();
14      reg clk,rst,d;
15      wire q;
16      d_ff uut(clk,rst,d,q);
17      always begin
18          #10 clk=~clk;
19      end
20      initial
21      begin
22          clk=0;rst=0;d=0;
23          #5 rst=1;
24          #5 rst=0;d=0;
25          #15 d=1;
26          #10 d=0;
27          #15 d=1;
28          #5 rst=1;
29          #5 rst=0; d=1;
30      $stop;
31      end
32  endmodule
```



Ln#

```
1  module sr_ff(clk,rst,s,r,q);
2      input clk,rst,s,r;
3      output reg q;
4      always@(posedge clk or posedge rst)
5      begin
6          if(rst==1)
7              q<=0;
8          else if (s==0 && r==0)
9              q<=q;
10         else if (s==0 && r==1)
11             q<=0;
12         else if (s==1 && r==0)
13             q<=1;
14         else if (s==1 && r==1)
15             q<=1'bx;
16     end
17 endmodule
18
19 module tb();
20     reg clk,rst,s,r;
21     wire q;
22     sr_ff uut(clk,rst,s,r,q);
23     always begin
24         #10 clk=~clk;
25     end
26     initial
27     begin
28         clk=0;rst=0;s=0;r=0;
29         #5 rst=1;
30         #5 rst=0;s=1; r=0;
31         #20 s=0; r=1;
32         #20 s=1; r=1;
33         #20 s=0; r=0;
34         #20 rst=1;
35         #5 rst=0;
36         $stop;
37     end
38 endmodule
39
```



Ln#

```
1  module t_ff(clk,rst,t,q);
2      input clk,rst,t;
3      output reg q;
4      always@(posedge clk or posedge rst)
5      begin
6          if(rst==1)
7              q<=0;
8          else if (t==0)
9              q<=q;
10         else
11             q<=~q;
12     end
13 endmodule
14 module tb();
15     reg clk,rst,t;
16     wire q;
17     t_ff uut(clk,rst,t,q);
18     always begin
19         #10 clk=~clk;
20     end
21     initial
22     begin
23         clk=0;rst=0;t=0;
24         #5 rst=1;
25         #5 rst=0;t=0;
26         #15 t=1;
27         #10 t=0;
28         #15 t=1;
29         #5 rst=1;
30         #5 rst=0; t=1;
31         $stop;
32     end
33 endmodule
34
```



Ln#

```
1  module jk_ff(clk,rst,j,k,q);
2      input clk,rst,j,k;
3      output reg q;
4      always@(posedge clk or posedge rst)
5      begin
6          if(rst==1)
7              q<=0;
8          else if (j==0 && k==0)
9              q<=q;
10         else if (j==0 && k==1)
11             q<=0;
12         else if (j==1 && k==0)
13             q<=1;
14         else if (j==1 && k==1)
15             q<=~q;
16     end
17 endmodule
18 module tb();
19     reg clk,rst,j,k;
20     wire q;
21     jk_ff uut(clk,rst,j,k,q);
22     always begin
23         #10 clk=~clk;
24     end
25     initial
26     begin
27         clk=0;rst=0;j=0;k=0;
28         #5 rst=1;
29         #5 rst=0;j=0; k=0;
30         #25 j=0; k=1;
31         #25 j=1; k=1;
32         #25 j=1; k=0;
33         #5 rst=1;
34         #25 rst=0; j=1; k=1;
35         $stop;
36     end
37 endmodule
38
```